UNIVERSIDAD CATÓLICA BOLIVIANA
LA PAZ

Carrera de Ingeniería Mecatrónica

ING. XAVIER ALEXIS MURILLO SANCHEZ

EMBEDDED SYSTEMS II

AGENDA

1. Bit Hex Unpack
2. TIVA GPIO configuration
3. TIVA Timers
4. Tiva ADC

BIT HEX UNPACK

- Is the process of manage addresses of different GPIO.
- In this case we will use this for TIVA GPIO definition.
- We can use the following process:

BIT HEX UNPACK

0x00 → First pack of bits

↓ Second pack of bits

0x00 = 0000 0000

BIT HEX UNPACK

0x00 → First pack of bits

↓ Second pack of bits

Bit 0 to 7

0x03 = 0000 0011

Enables pin 0 and 1

UNIVERSIDAD CATÓLICA BOLIVIANA LA PAZ
Carrera de Ingeniería Mecatrónica
IMT UCB

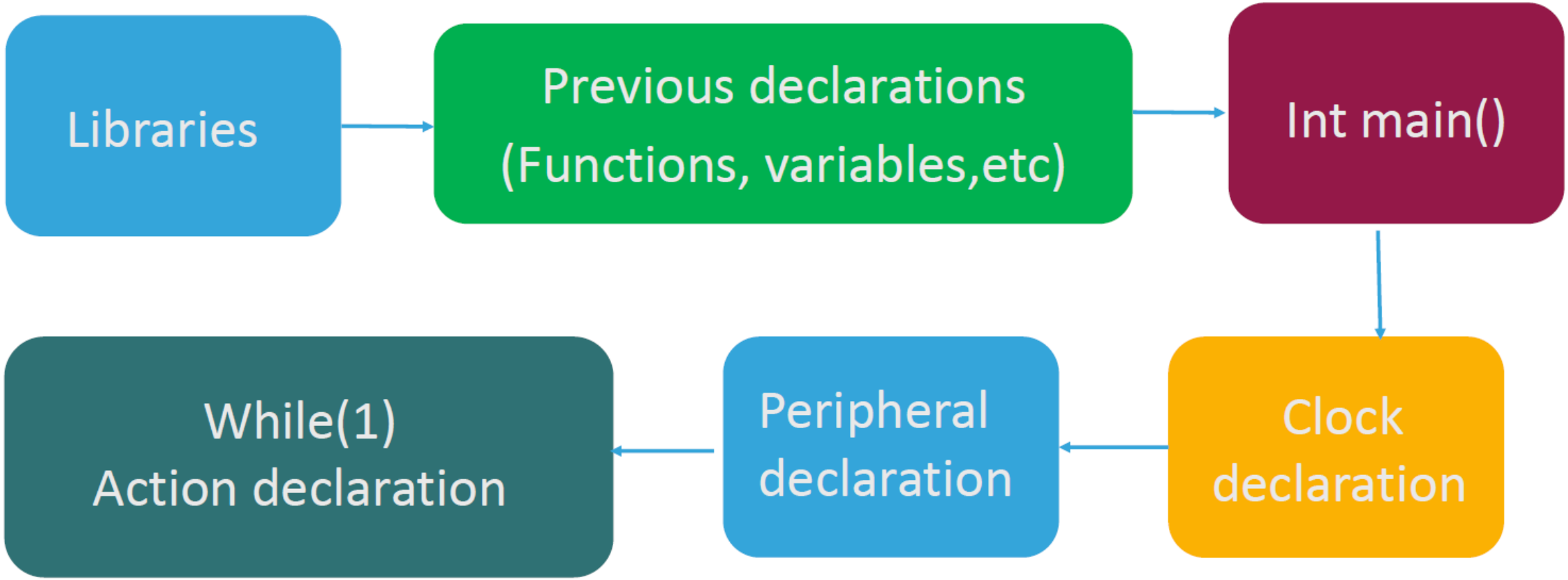
BIT HEX UNPACK

- As we saw each bit represents a pin value going from 0 to 7.
- We have to read the values from RIGHT to LEFT
- The max value would be 0xff.

UNIVERSIDAD CATÓLICA BOLIVIANA LA PAZ
Carrera de Ingeniería Mecatrónica

Questions?

PROJECT IN TIVA



UNIVERSIDAD CATÓLICA BOLIVIANA LA PAZ
Carrera de Ingeniería Mecatrónica

TIVA GPIO CONFIGURATION

- TIVA is especially complicate to understand and learn in comparison to some previous boards.
- However, with the right tools we can improve our skills
- We need: Driverlib, TIVA datasheet, TIVA peripheral driver library, TIVA examples.

TIVA GPIO CONFIGURATION

- TIVA can use General Purpose Input/Output for different uses like:
  - Read sensor values
  - Send output signal to some pins
  - For read values we use an ADC peripheral
  - For write values we can send digital signals or use PWM
  - Let’s see how to manage a GPIO digital output

TIVA GPIO CONFIGURATION

```
//Libraries needed for TIVA functions
#include <stdint.h>
#include <stdbool.h>
#include "inc/hw_memmap.h"
#include "driverlib/debug.h"
#include "driverlib/gpio.h"
#include "driverlib/sysctl.h"
```

Libraries

```
//*****************************************************************************
//
// The error routine that is called if the driver library encounters an error.
//
//*****************************************************************************
#ifdef DEBUG
void
__error__(char *pcFilename, uint32_t ui32Line)
{
    while(1);
}
#endif

int main(void)
{
    volatile uint32_t ui32Loop;

    SysCtlClockFreqSet((SYSCTL_XTAL_25MHZ | SYSCTL_OSC_MAIN | SYSCTL_USE_PLL | SYSCTL_CFG_VCO_480),120000000); // Eables system clock

    SysCtlPeripheralEnable(SYSCTL_PERIPH_GPION); //Enables peripheral N
    SysCtlPeripheralEnable(SYSCTL_PERIPH_GPIOF); //Enables peripheral F
```

Clock configuration

Peripheral enabling

PROJECT IN TIVA

```
SysCtlPeripheralEnable(SYSCTL_PERIPH_GPION); //Enables peripheral N
SysCtlPeripheralEnable(SYSCTL_PERIPH_GPIOF); //Enables peripheral F

// Check if the peripheral access is enabled.
while(!SysCtlPeripheralReady(SYSCTL_PERIPH_GPION))
{
}

while(!SysCtlPeripheralReady(SYSCTL_PERIPH_GPIOF))
{
}

GPIOPinTypeGPIOOutput(GPIO_PORTN_BASE, 0x03); //enables pin 0 and 1
GPIOPinTypeGPIOOutput(GPIO_PORTF_BASE, 0x11); //enables pin 0 and 4
```

Pin enabling (using hex direction)

UNIVERSIDAD CATÓLICA BOLIVIANA LA PAZ
Carrera de Ingeniería Mecatrónica

PROJECT IN TIVA

Program execution

```
    // Loop forever.
    while(1)
    {
        // Turn on the LEDs.
        GPIOPinWrite(GPIO_PORTN_BASE,0x03,0x02);  //set periph base,pins, values
        GPIOPinWrite(GPIO_PORTF_BASE,0x11,0x01);  //set periph base,pins, values

        // Delay for a bit.
        for(ui32Loop = 0; ui32Loop < 20000000; ui32Loop++)
        {
        }

        // Turn off the LEDs.
        GPIOPinWrite(GPIO_PORTN_BASE,0x03,0x01);
        GPIOPinWrite(GPIO_PORTF_BASE,0x11,0x10);

        // Delay for a bit.
        for(ui32Loop = 0; ui32Loop < 20000000; ui32Loop++)
        {
        }

    }
}
```

PROJECT IN TIVA

Program execution

```
//Set GPIO as input/output
GPIOPinTypeGPIOOutput(GPIO_PORTN_BASE, 0x03); //enables pin 0 and 1
GPIOPinTypeGPIOOutput(GPIO_PORTF_BASE, 0x11); //enables pin 0 and 4
//GPIO as Input
GPIOPinTypeGPIOInput(GPIO_PORTJ_BASE, 0x00);
//Configures input current strenght and pull type
GPIOPadConfigSet(GPIO_PORTJ_BASE,GPIO_PIN_0|GPIO_PIN_1,GPIO_STRENGTH_2MA, GPIO_PIN_TYPE_STD_WPU);
```

As you can see, we use the word Input, and we need to declare the pull type (see documentation)

UNIVERSIDAD CATÓLICA BOLIVIANA LA PAZ
Carrera de Ingeniería Mecatrónica

PROJECT IN TIVA

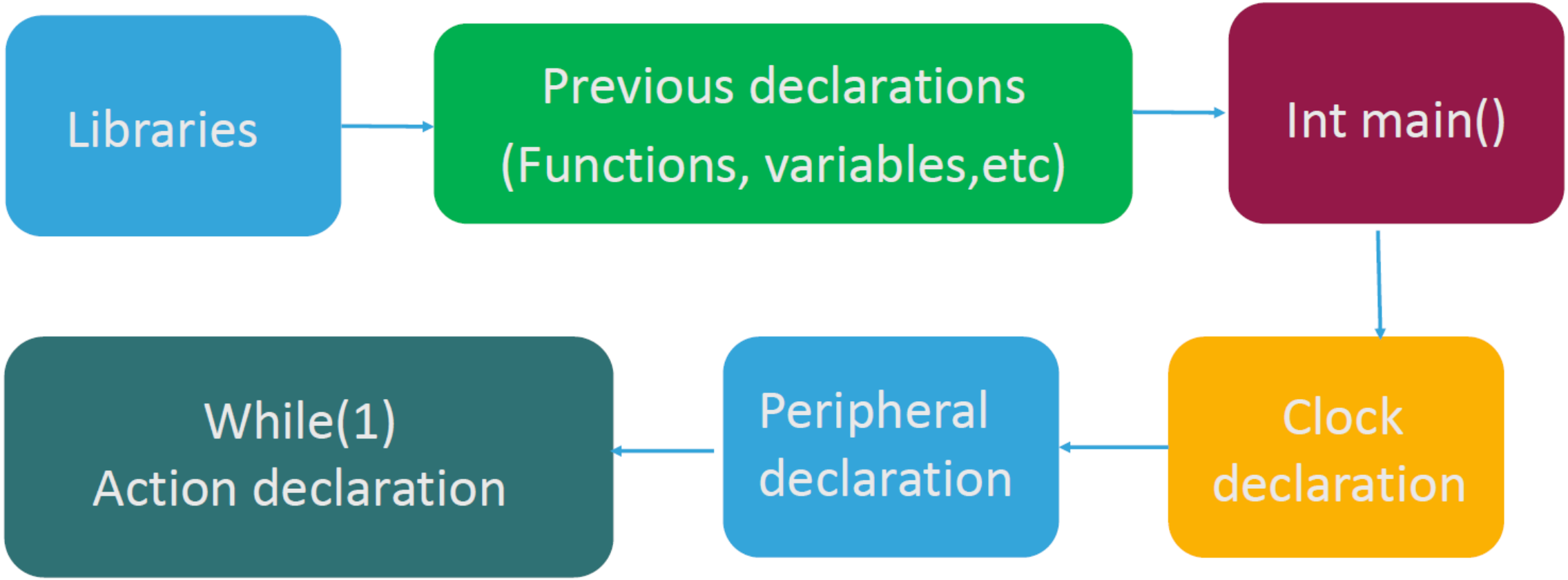
Program execution

```
if (GPIOPinRead(GPIO_PORTJ_BASE, GPIO_PIN_0) == 0)
 {
  button ++;
 }
```

We can read the value of the input depending of the pull type (pull up = 0 and pull down= GPIO)

UNIVERSIDAD CATÓLICA BOLIVIANA LA PAZ
Carrera de Ingeniería Mecatrónica

PROJECT IN TIVA



UNIVERSIDAD CATÓLICA BOLIVIANA LA PAZ
Carrera de Ingeniería Mecatrónica

TIVA GPIO CONFIGURATION

- Is necessary to enable the peripheral for the GPIO.
- We define the pin type as input or output (in this case output)
- As we are using outputs we have to write the pin value.
- The value to write is the GPIO value (NOT 1) or 0.

TIVA TIMERS

- Timers are used to generate operations according to some events during the time.
- Generally, we will generate interruptions to the main routine when a timer is executed.
- We can use delay to interrupt all the program execution and timers to change the main routine for a sub routine.

TIVA TIMERS

```
uint32_t FS= 120000000*2;
void timer0A_handler(void);
uint8_t switch_state = 0;
uint32_t button = 0;
int main(void)
{
    //Clock Configuration
    SysCtlClockFreqSet((SYSCTL_XTAL_25MHZ | SYSCTL_OSC_MAIN | SYSCTL_USE_PLL | SYSCTL_CFG_VCO_480),120000000); // Eables system clock
    //Enable peripherals
    SysCtlPeripheralEnable(SYSCTL_PERIPH_GPION); //Enables peripheral N
    SysCtlPeripheralEnable(SYSCTL_PERIPH_GPIOF); //Enables peripheral F
    SysCtlPeripheralEnable(SYSCTL_PERIPH_GPIOJ);
    //Enable Timer Peripheral
    SysCtlPeripheralEnable(SYSCTL_PERIPH_TIMER0);
```

We must enable the Timer peripheral to use it.

PROJECT IN TIVA

```
//Set timer
TimerConfigure(TIMER0_BASE,TIMER_CFG_PERIODIC);
//Set the cout time for Timer
TimerLoadSet(TIMER0_BASE, TIMER_A, FS);
//Enable processor interrupts
IntMasterEnable();
//Enable Interrupt
IntEnable(INT_TIMER0A);
//Enable timer A interrupt
TimerIntEnable(TIMER0_BASE, TIMER_TIMA_TIMEOUT);
//Enable the timer
TimerEnable(TIMER0_BASE,TIMER_A);
```

This is the main routine to deine a timer, configure it and enable it.

FS is the frequence sample, it is used to get the timer intervals:

timerFreq=clockfreq/FS

UNIVERSIDAD CATÓLICA BOLIVIANA LA PAZ
Carrera de Ingeniería Mecatrónica

PROJECT IN TIVA

```
void timer0A_handler(void)
{
    switch_state ++;
    //Changing the timer delay
    if (GPIOPinRead(GPIO_PORTJ_BASE, GPIO_PIN_0) == 0)
     {
      button ++;
     }
    if(button >0)
    {
          FS =120000000*4;
          TimerLoadSet(TIMER0_BASE, TIMER_A, FS);
    }
    //Clear timer
    TimerIntClear(TIMER0_BASE, TIMER_A);
    // Sequence 1.
    if(switch_state % 2 != 0)
    {
         GPIOPinWrite(GPIO_PORTN_BASE,0x03,0x02);  //set periph base,pins, values
         GPIOPinWrite(GPIO_PORTF_BASE,0x11,0x01);  //set periph base,pins, values
    }
    // Sequence 2.
    if(switch_state % 2 == 0)
    {
          GPIOPinWrite(GPIO_PORTN_BASE,0x03,0x01);
```

The timer uses a external function to execute the sub routine. This routine will stop the main routine and depending if the timer is periodic or one shot will keep executing

PROJECT IN TIVA

```
//*****************************************************************************
//
// The entry point for the application.
//
//*****************************************************************************
extern int main(void);
extern void timer0A_handler(void);
//*****************************************************************************
```

We have to declare the timer function in the startup file of the project.

```
3    IntDefaultHandler,                    // ADC Sequence 0
4    IntDefaultHandler,                    // ADC Sequence 1
5    IntDefaultHandler,                    // ADC Sequence 2
6    IntDefaultHandler,                    // ADC Sequence 3
7    IntDefaultHandler,                    // Watchdog timer
8    timer0A_handler,                    // Timer 0 subtimer A
9    IntDefaultHandler,                    // Timer 0 subtimer B
0    IntDefaultHandler,                    // Timer 1 subtimer A
1    IntDefaultHandler,                    // Timer 1 subtimer B
```

THANK YOU FOR YOUR ATTENTION!